ASAP: A DiSaggregated and Asynchronous Inference System for MoE Prefill

Weiwei Chen, Shuang Chen,* Lele Li, Qiang Hu, Han Li, Xin Ye, Ming Yan, Zhibin Yu

Huawei

Abstract

Mixture-of-Experts (MoE) models have become the de facto standard for scaling large language models. To maintain computational efficiency, modern MoE serving systems typically employ a hybrid parallelism strategy, combining Data Parallelism (DP) for attention stages with Expert Parallelism (EP) for MoE stages. However, this design necessitates frequent global synchronization barriers between attention DP groups and experts. In online serving, significant variance in request arrival rates and sequence lengths inherently leads to *DP imbalance*, causing severe synchronization stalls that degrade Time-to-First-Token (TTFT) and system throughput.

We present ASAP, an asynchronous inference system specifically designed to accelerate the prefill phase of MoE models. ASAP disaggregates the attention and MoE stages and implements a fully asynchronous execution pipeline. This is achieved through a suite of specialized asynchronous communication primitives and four coordinated optimizations across request scheduling and model execution, which collectively dismantle global synchronization barriers. We implement and evaluate ASAP on CloudMatrix384 super-nodes, demonstrating that it improves SLO-compliant prefill throughput by 90% compared to state-of-the-art synchronous serving solutions.

1 Introduction

Background. Large-scale Mixture-of-Experts (MoE) architectures have become the de facto paradigm for serving high-capacity models efficiently. By selectively activating a sparse subset of experts per token [15, 16, 17, 35, 39, 41, 42], MoEs decouple model capability from computational cost. For example, DeepSeek-V3 [13] achieves a total scale of 671B parameters while restricting active parameters to only 37B per token.

Since the vast majority of these parameters reside in expert weights, Expert Parallelism (EP) is commonly applied to the MoE stage, distributing experts across an array of accelerators. A wide EP configuration fundamentally reduces the memory footprint per device, directly alleviating the severe memory bottlenecks typical in LLM inference. Specifically, a DeepSeek-V3 prefill instance [13] utilizes an EP size of 32, distributed across 32 GPUs over 4 compute nodes.

However, accommodating this broad EP scale introduces architectural friction for the attention stage, which must adopt a hybrid of Tensor Parallelism (TP) and Data Parallelism (DP). Due to the prohibitive inter-device communication overhead incurred by large TP, the TP degree is strictly bounded (typically ≤ 8). As a result, the system configures the DP size as EP/TP to process concurrent request batches [27, 40]. Consequently, the aforementioned DeepSeek-V3 instance enforces a TP size of 8 and a DP size of 4 during its attention phase.

Problem. Despite the theoretical efficiency of these hybrid parallelism strategies, our profiling reveals severe resource underutilization and inflated inference latencies during the compute-intensive prefill phase of production MoE serving. This inefficiency stems from a fundamental execution mismatch: the **inevitable DP imbalance** coupled with the **rigid synchronization barriers** inherent in state-of-the-art inference systems.

As depicted in Figure 1a, heterogeneous requests are independently batched and routed to distinct attention DP groups. However, because the subsequent MoE stage operates over a globally shared expert pool, these isolated DP groups are forced to synchronize at strict barriers before and after every MoE layer. This lockstep execution paradigm creates a severe **straggler effect**, where the entire system stalls until the slowest attention DP group or the most congested expert completes its task.

In production deployments, online inference clusters are subjected to highly stochastic request arrivals, extreme sequence length skew, unpredictable prefix cache hit rates, etc [7, 49]. Such heterogeneity results in imbalanced workload (thus inconsistent latency) across attention DP groups, a phenomenon we term *DP Imbalance* [31].

Related Work. Existing literature has extensively explored expert load balancing [11, 13, 34], yet the performance degradation caused by DP imbalance has been significantly underestimated in the context of online MoE serving. Because production LLM serving is characterized by highly stochastic arrivals and heavy-tailed sequence length distributions [7, 49], achieving perfect attention DP balancing is practically impossible. On another front, recent efforts in long-context acceleration [9, 14, 21, 23, 24, 26, 36, 45, 47, 54] successfully tackle

*Corresponding author. Email: chenshuang0804@gmail.com

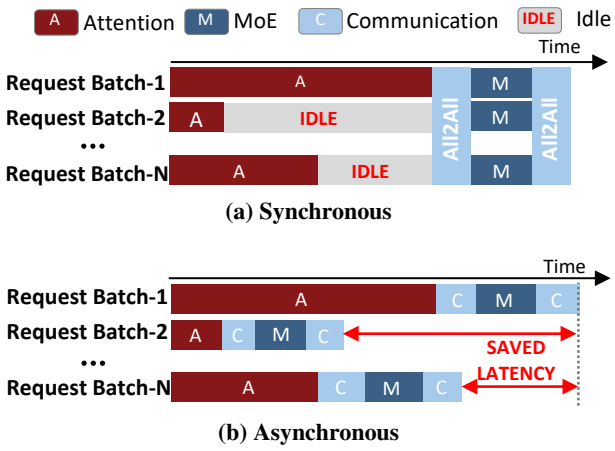


Figure 1: Comparison between current synchronous and ideal asynchronous execution under heterogeneous request batches. Synchronous execution (Figure 1a) requires all request batches (launched on different attention DP groups) to synchronize before the MoE computation. An ideal asynchronous execution pipeline (Figure 1b) eliminates the barrier, and allows faster request batches to progress at their own pace.

super-long requests, but remain oblivious to the execution bubbles caused by heterogeneous short-to-medium requests (e.g., prompts with < 32K tokens). Our characterization in Section 2.2.1 shows that DP imbalance does not necessarily come from super-long requests. Co-executing various short-to-medium requests in a synchronous inference system still stalls the pipeline and degrades overall throughput.

Motivation. Figure 1b illustrates an ideal, barrier-free, asynchronous execution paradigm. By eliminating global synchronization, heterogeneous requests are enabled to truly "flow" at their own pace without waiting for or delaying others. This approach effectively minimizes resource idling, which simultaneously reduces inference latency and improves overall throughput.

Challenges. Transitioning from synchronous to asynchronous MoE serving presents two fundamental architectural hurdles:

1. **Decoupling Execution Dependencies with Asynchronous Communication Primitives:** Traditional systems tightly couple Attention and MoE stages on the same devices, enforcing a lock-step execution that precludes overlapping different computation stages. Transitioning to an Attention-MoE decoupled architecture requires not only physical resource disaggregation, but also a new class of asymmetric and asynchronous communication primitives. Current collective operations (e.g., Peer-to-Peer, All-to-All) are designed for synchronous blocking bulk transfers; the lack of efficient primitives that support fine-grained, non-blocking token routing is a primary reason why fully asynchronous MoE inference remains elusive.
2. **Maintaining Compute Efficiency under Fragmented and Out-of-Order Execution:** Asynchrony introduces "fragmented" execution; instead of one massive batch, MoE devices receive independent, smaller batches, risking a drop in arithmetic intensity. Furthermore, DP imbalance forces MoE devices to execute layers out-of-order, where the specific layer ID is only resolved at runtime. This dynamic execution pattern significantly increases kernel dispatch overhead from the host CPU, potentially introducing more execution bubbles than the gains obtained from asynchrony.

Our Work. We present ASAP, a high-performance inference system that realizes fully asynchronous MoE prefill. ASAP disaggregates attention and MoE stages onto separate devices, interconnected via asymmetric, asynchronous communication primitives to enable non-blocking data transfers. The key is to create a dedicated shared memory buffer on each device that is visible to all devices and acts as a "super-hub" between senders and receivers. This is the foundation of ASAP's asynchronous execution pipeline.

To fully leverage this asynchronous architecture and mitigate the efficiency risks of asynchrony, ASAP integrates four key runtime optimizations: (1) **Length-aware batching** to preserve MoE compute intensity; (2) **Dual-batch interleaving** to maximize attention device duty cycles; (3) **Communication-computation overlapping** via a triple-stream design to maximize hardware utilization; and (4) A layer-oblivious *MoE Super Kernel* that enables **bubble-free kernel dispatching**, allowing out-of-order execution without host-side bottlenecks.

In summary, we make the following contributions:

1. To the best of our knowledge, we are the first to build an asynchronous execution pipeline for online inference.
2. We systematically analyze the prefill phase of MoE serving. We identify an inherent *DP imbalance* problem in synchronous systems, causing severe resource idling and limited prefill performance.
3. We design ASAP, the first asynchronous MoE inference system that dismantles global synchronization barriers to enable independent execution of request batches.
4. We implement and evaluate ASAP on CloudMatrix384 [56] supernode. Extensive evaluation results show that ASAP improves SLO-compliant prefill throughput by 90% compared to state-of-the-art synchronous baselines.

2 Background and Motivation

In this section, we provide the necessary background on MoE serving, and present a series of characterization studies to motivate the need for a barrier-free, asynchronous system to optimize MoE prefill performance.

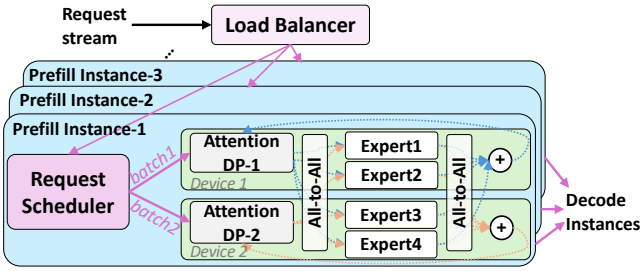
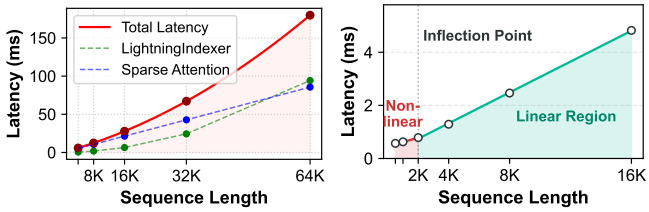


Figure 2: A typical MoE serving cluster consists of a number of serving instances. A user request is routed to a designated instance through a cluster-level load balancer. It is then aggregated into a request batch with other incoming requests, and dispatched to a specific attention DP group by the instance-level request scheduler.



(a) Attention (b) MoE

Figure 3: Latency scaling with increasing sequence length.

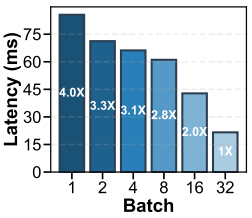


Figure 4: Impact of batch size on attention latency under a fixed total sequence length of 32k tokens.

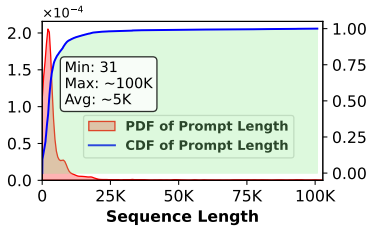


Figure 5: CDF and PDF of sequence lengths from a production dataset on *Huawei Cloud*, demonstrating extreme request variance.

2.1 LLM Inference Preliminaries

Prefill and Decode Phases. In LLM serving, a user request first undergoes the prefill phase to generate the initial token, followed by the decode phase to generate subsequent tokens autoregressively [8, 51]. Inference latency is primarily characterized by Time-to-First-Token (TTFT) for prefill and Time-Per-Output-Token (TPOT) for decode. Since TTFT reflects how long the user has to wait before getting responses, its service-level objective (SLO) is usually within a few seconds [10, 19]. In contrast, SLO of TPOT is usually at tens of milliseconds, to keep up with the user's reading speed.

Given their divergent profiles—prefill being compute-bound and decode being memory-bound—it has become the standard practice to adopt prefill-decode disaggregation in production-grade inference frameworks [22, 37, 48, 53].

MoE Architectures. MoE models replace monolithic feed-forward networks (FFN) with a set of sparse, specialized sub-networks (experts). By activating only a fraction of parameters per token, MoE enables massive model scaling without a proportional increase in inference FLOPs. This architecture has rapidly become the de facto standard for state-of-the-art LLMs, as evidenced by the recent releases of DeepSeek-V3 [13], Kimi-K2 [42], and Qwen3-MoE [50].

MoE Serving Infrastructure. Figure 2 illustrates the typical architecture of an MoE serving cluster that is composed of a number of serving instances. Prefill-decode disaggregation is adopted, and we focus on illustrating the prefill phase (applicability to the decode phase is discussed in Section 4.4). Upon the ingress of a user request, a cluster-level load balancer routes it to a designated prefill instance, typically following a round-robin policy. Within the assigned instance, the request is aggregated into a batch, and dispatched to a specific attention DP group by the instance-level request scheduler. After the first token is generated, the request along with its KV cache is dispatched to a designated decode instance to generate subsequent tokens.

To accommodate the massive parameter scale of MoE models, each serving instance typically employs wide Expert Parallelism (e.g., $EP = 32$ in DeepSeek V3 [13]) to distribute experts across multiple accelerators, thereby alleviating the memory pressure per device [13, 55]. For the attention stage, Tensor Parallelism (TP) is usually applied to reduce attention latency. However, due to the large volume of communication, TP is generally limited to values of no more than 8. To match the large size of the EP, the attention stage also adopts Data Parallelism (i.e., $DP = EP/TP$), as shown in Figure 2, .

> In this paper, we focus on optimizing performance of the prefill instances in MoE serving with a combination of attention DP and expert EP.

2.2 Characterization of MoE Inference

To thoroughly analyze the runtime characteristics of the MoE prefill phase, we deploy DeepSeek-V3.2 [30], a state-of-the-art open-sourced MoE model released in Dec 2025, on 32 Ascend NPUs [56]. Our setup follows the baseline configuration described in the DeepSeek technical report [13], utilizing a hybrid parallelism strategy of $EP = 32$, $DP = 8$, and $TP = 4$. Comprehensive specifications of the model and hardware platform are detailed in Section 5.1.

2.2.1 Attention

The attention stage is compute-intensive for the prefill phase. Figure 3a plots attention latency as a function of sequence length s for a batch size of one. It shows that the latency

exhibits a quadratic correlation with s. Although DeepSeek-V3.2 incorporates DeepSeek Sparse Attention (DSA) [30] where the sparse attention computation itself scales linearly, $O(s)$, the associated lightning indexer retains an $O(s^2)$ complexity. The combined effect of these operations dictates the overall quadratic scaling.

In production, request schedulers dynamically bundle multiple requests into a single batch to maximize hardware utilization. However, batching to a uniform total sequence length does not yield deterministic execution time. Figure 4 demonstrates the attention latency for a fixed total budget of 32k tokens across varying batch sizes. A batch size of 32 (1k tokens per request) compared to a batch size of 1 (32k tokens per request) reveals a stark latency disparity of up to 4.2×. This significant variance arises because the attention complexity for a batch of n requests is governed by the sum of the squares of individual sequence lengths, $O(\sum_{i=1}^{n} s_i^2)$ [46], rather than the square of their total sum.

> Attention latency scales quadratically with individual request lengths. Consequently, the aggregate token count of a batch is a poor predictor of execution time.

2.2.2 MoE

Conversely, the latency of the MoE stage is dictated primarily by the aggregate token count within a batch. Figure 3b shows the scaling of MoE latency with s under perfect expert load balancing. It reveals that MoE latency remains initially flat, transitioning to linear scaling only after surpassing an inflection point (approximately 2k tokens). This plateau occurs because the MoE stage is strictly memory-bound at low token count; the hardware must load massive expert weights from memory while performing relatively few computation operations. As sequence length grows, arithmetic intensity improves, pushing the system into a compute-bound regime. While the exact inflection point varies across hardware platforms and model configurations, this dual-regime scaling behavior—an initial memory-bound plateau followed by a linear compute-bound phase—is an intrinsic characteristic of MoE computation.

> The MoE stage requires a minimum aggregate token count to saturate hardware compute units and transition from a memory-bound plateau to a high-efficiency linear regime.

2.3 DP Imbalance in Online Serving

In production environments, inference clusters are subjected to highly stochastic request arrivals and extreme heterogeneity in sequence lengths, prefix cache hit rate, etc [7, 49]. Figure 5 visualizes the sequence length distribution from real-world

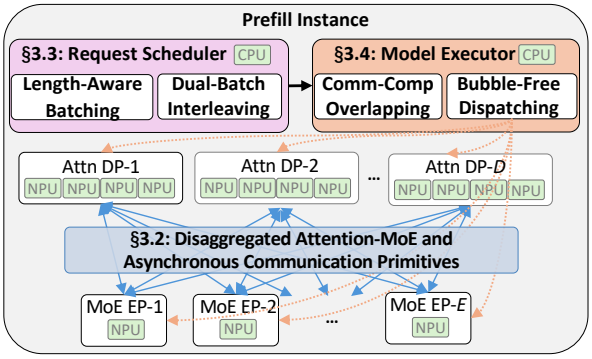


Figure 6: ASAP Overview.

Huawei Cloud [1] traces. While the mean prompt length sits at 5k tokens, the distribution exhibits a massive variance—ranging from a mere 31 tokens to a heavy tail extending up to 100k tokens. As established in Section 2.2.1, excluding the impact of prefix cache, simply matching the total token count across attention DP groups is still fundamentally inadequate. Perfect load balancing would require that the sum of squared sequence lengths ($O(\sum s_i^2)$) precisely match, a constraint that is virtually impossible to satisfy dynamically.

> The combination of $O(s^2)$ attention complexity, heavy-tailed sequence length, and other factors (e.g., prefix cache) inherently causes latency variations across attention DP groups. Such DP imbalance leads to execution bubbles and resource idling in synchronous systems.

3 ASAP Design

In this section, we present ASAP, an asynchronous inference framework designed to accelerate the prefill phase of MoE models. By replacing the rigid barriers of conventional synchronous systems with a fully decoupled and asynchronous execution pipeline, ASAP targets minimizing resource idling and maximizing prefill throughput under SLO constraints. To facilitate the subsequent discussion, Table 1 lists all the symbols, definitions, and the representative configurations (detailed in Section 5.1) used throughout this paper.

3.1 Overview

Figure 6 provides an overview of the ASAP architecture. ASAP first adopts a disaggregated design by partitioning attention and MoE stages onto separate hardware devices. Specifically, the attention stage occupies $D \times T$ NPUs, while the MoE stage is deployed across E NPUs , resulting in a total hardware footprint of $D \times T + E$ devices per prefill instance.

To bridge these disaggregated components, we develop a suite of asynchronous communication primitives that support

[1] A token service in production. The service and company names are kept anonymous for double blindness.

Table 1: Symbol Definition. Representative configurations are detailed in Section 5.1.

Symbol	Representative Configurations	Definition
D	4	Number of attention DP groups
T	4	TP in each attention DP group
E	16	EP
E_{total}	256	Number of experts in the model
K	8	TopK in the model
L	61	Number of layers in the model
H	7168	Hidden size of the model
$Dsize$	16 bits	Data size
S	32k	Maximum batch sequence length

non-blocking 1-to-N/N-to-1 data transfers. This structural decoupling forms the foundation of our barrier-free inference pipeline, allowing the attention and MoE stages to progress independently without global synchronization.

Based upon this foundation, ASAP incorporates four key mechanisms within the request scheduler and model executor to maintain high hardware utilization:

1. **Length-aware batching** to preserve MoE computational efficiency even under fragmented workloads;
2. **Dual-batch interleaving** to prevent attention devices idling during active MoE processing;
3. **Communication-computation overlapping** via a triple-stream design to maximize hardware resource utilization;
4. **Bubble-free kernel dispatching** to eliminate host-side dispatching overheads during out-of-order execution.

3.2 Asynchronous Communication Primitives

Conventional peer-to-peer and all-to-all collectives are inherently blocking, requiring explicit handshaking between senders and receivers. To eliminate this bottleneck, we implement a **distributed shared-memory abstraction**, where each device allocates a globally visible buffer as a decentralized communication hub. These buffers are **statically allocated** during initialization and persist throughout the framework's lifetime. Table 2 details the buffer structure and memory footprint on each device.

Under this model, senders write to the target buffer and immediately resume computation without waiting for receiver acknowledgment. Similarly, receivers independently retrieve data from these buffers upon completing local tasks. We leverage this abstraction and develop four specialized primitives, two for asynchronous dispatch and another two for asynchronous combine. These are designed to replace their synchronous counterparts, both of which are traditionally implemented as blocking all-to-all collectives.

Table 2: Shared Buffer Structure. Example size is derived from the representative configurations specified in Table 1.

	ID	Memory Size	Example Size	Description
Attention	①	$K \times S/T$	64KB	Expert IDs
	②	$H \times K \times S \times Dsize/T$	0.9GB	Expert results
	③	$E/8$	<1KB	E-bit bitmap flag
MoE	①	$D \times T \times E_{total}/E$	<1KB	Token metadata
	②	$D \times H \times K \times S \times Dsize$	14GB	Tokens (hidden states)
	③	$D \times T/8$	<1KB	D T-bit bitmap flags

3.2.1 Asynchronous Dispatch

This occurs between an attention device and a set of MoE devices following each attention layer. Each MoE device allocates a shared buffer partitioned into D regions (one per attention DP group), with each region further subdivided into T rows (one per TP member within that DP group).

As illustrated in Figure 7a, the shared buffer consists of three sub-regions: ① token metadata stores token counts per expert (E_{total}/E integers; ② token payload stores the actual token hidden states; and ③ a T-bit bitmap indicating data readiness for each TP member.

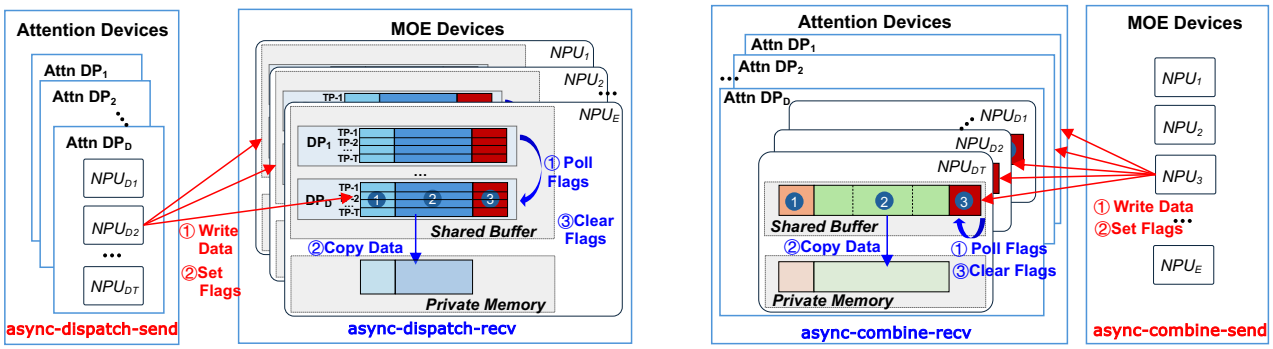
Execution Flow: An attention device NPU_{ij} invokes `async-dispatch-send` to write its local tokens into the j-th row of the i-th region in each target MoE device. To ensure data integrity, the sender employs a backpressure mechanism: if a target flag is already set, the write blocks until the receiver clears the flag. Once the transfer completes, the sender sets its corresponding bit in the bitmap (③).

On the receiver side, the MoE device executes `async-dispatch-recv` by polling the bitmaps across all regions. When all T flags for a specific DP_i are set, the MoE device migrates the token metadata (①) and hidden states (②) to its private memory and clears the bitmap ③ to acknowledge availability for subsequent transfers.

3.2.2 Asynchronous Combine

This retrieves computed expert results from multiple MoE devices back to their originating attention DP group after each MoE layer. The attention device's shared buffer comprises: ① expert IDs for token-to-expert mapping; ② expert results partitioned into E segments (one per MoE device); and ③ an E-bit bitmap flag that indicates result arrival from each MoE device.

Execution Flow: Upon completing the computation for DP_i, an MoE device invokes `async-combine-send` to write expert results into the shared buffer of the T attention devices belonging to DP_i, and sets the corresponding completion bit. The attention device executes `async-combine-recv`, monitoring the bitmap flags. Once all activated expert results are received, the attention device migrates the payload to its private memory and clears the flags for the next iteration.



(a) Asynchronous Dispatch **(b) Asynchronous Combine**

Figure 7: Asynchronous communication primitives and the shared buffer design. The buffer structure on each device is detailed in Table 2. Senders write data and set flags into the target device's shared buffer, returning to computation without synchronization. Receivers monitor these flags to retrieve and clear data as their own tasks progress.

3.3 Request Scheduler

While the asynchronous framework eliminates synchronization stalls, it introduces new challenges for maintaining high hardware utilization. Specifically, two primary issues arise. First, in a decoupled architecture, MoE devices process request batches sequentially rather than concurrently across all D attention DP groups. This leads to reduced token density per MoE invocation, which may compromise computational efficiency. Second, resource disaggregation creates execution gaps; attention devices become idle while their respective batches are in the MoE stage.

To address these challenges, we propose two scheduling optimizations as follows.

3.3.1 Length-Aware Batching

As characterized in Section 2.2.2, the MoE stage requires a minimum token count (e.g., 2k tokens) to reach the compute-bound linear regime. Because the MoE stage processes batches from independent attention DP groups serially, maintaining a high per-batch token count is critical to saturate hardware utilization in an asynchronous environment. Our resource scheduler enforces a policy that aggregates requests into batches exceeding the inflection point in Figure 3b (e.g., 2k tokens in our setup). Notably, since ASAP allows DP groups to progress independently, the batching algorithm is significantly simplified as it no longer needs to optimize for load balancing across attention DP groups.

3.3.2 Dual-Batch Interleaving

To eliminate attention device idling during MoE execution, we porpose dual-batch interleaving. Rather than scheduling a single batch immediately upon reaching the inflection point, ASAP waits for additional requests to form a dual-batch pair, and co-schedule the two batches to an idle DP group.

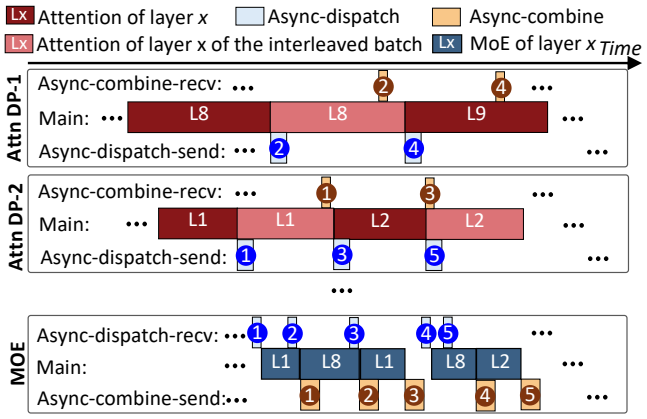
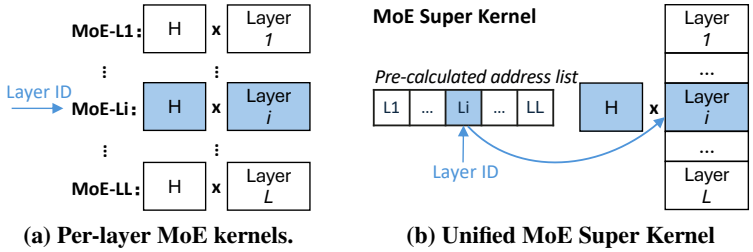


Figure 8: Dual-batch interleaving and the triple-stream design for communication-computation overlapping. For each attention DP group, two request batches are co-scheduled to interleave their layer-wise attention processing together. All devices employ a triple-stream design to further overlap asynchronous communication primitives with the main active computation kernels.

As illustrated in Figure 8, the attention DP group interleaves the layer-wise processing of the two batches (the dark red and light red bars). When one batch is offloaded to the MoE stage, the attention devices immediately commence processing for the other batch of the same layer.

The efficiency of this interleaving depends on the relative latencies of the attention and MoE stages.

- **Attention-limited cases**: If attention latency significantly exceeds MoE latency, each batch may experience queueing delays on attention devices. As shown in Figure 3, for sequences exceeding 16k tokens, MoE latency accounts for less than 15% of attention latency. Consequently, we disable interleaving for these long sequences, allowing them to exclusively occupy an attention DP group to minimize queuing from dual-batch interleaving.



(a) Per-layer MoE kernels. (b) Unified MoE Super Kernel

Figure 9: Comparison between standard per-layer MoE kernels and the proposed MoE Super Kernel. Block "H" denotes the tensor of hidden states, Block "Layer i" denotes the tensor of the expert weights of the i^{th} layer.

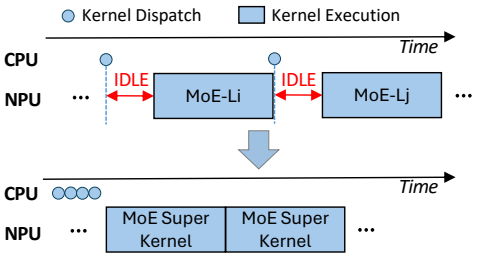


Figure 10: The layer-oblivious design of the MoE Super Kernel allows ahead-of-time dispatching, eliminating device idling between kernels.

- **MoE-limited cases**: Conversely, if attention latency of the interleaved batch is shorter than MoE latency, then idling of attention devices will persist. However, with length-aware batching, attention execution typically surpasses several milliseconds, comfortably overlapping with the sub-millisecond to few-millisecond MoE latency.

3.4 Model Executor

The model executor serves as the core engine for inference execution. It orchestrates parallelism strategies, interfaces with the underlying hardware, and dispatches NPU kernels. Two primary runtime optimizations are proposed to minimize resource idling and maximize system throughput.

3.4.1 Communication-Computation Overlapping

Communication typically accounts for 10% to 20% of total execution latency [18]. To effectively mask communication overhead, ASAP employs a **triple-stream concurrency model** on each device (Figure 8). A dedicated compute stream executes the model's forward pass, while two independent communication streams handle asynchronous data transfers, ensuring that the hardware compute units remain saturated.

Specifically, the main compute stream of each attention device interleaves the layer-wise forward passes of the dual batches, while that of each MoE device processes MoE layers out-of-order. Each communication stream handles one of the four asynchronous primitives. By mapping these independent tasks to isolated hardware streams and allocate dedicated hardware resources to each stream, ASAP approaches non-interfering execution (detailed in Section 4).

3.4.2 Bubble-Free Kernel Dispatching

As shown in Figure 6, the MoE stage is a *shared* by multiple attention DP groups. Due to asynchronous processing, the MoE module executes layers in a nondeterministic, out-of-order manner. For instance, it may interleave the execution of Layer 8 from one DP group with Layer 1 from a newly scheduled batch in another DP group (Figure 8). This layer execution order is highly nondeterministic, driven entirely by the independent progress of each attention DP group.

This dynamic execution order poses a significant challenge for kernel dispatching. Typically, host CPUs launch NPU kernels asynchronously in advance, effectively hiding the host-side dispatch overhead (e.g., argument preparation, memory allocation, tiling preparation, runtime API calls) behind active accelerator execution. However, because the exact layer ID to be executed next by the MoE stage is resolved dynamically at runtime, kernels cannot be pre-launched. This forces the CPU dispatch overhead—ranging from tens to hundreds of microseconds—onto the critical path (Figure 10). Given that individual MoE kernel execution spans merely sub-millisecond to a few milliseconds, this dispatch overhead constitutes a non-negligible performance bubble.

To eliminate these execution bubbles, we redesign the standard per-layer MoE kernel (Figure 9a) into a unified layer-oblivious **MoE Super Kernel** (Figure 9b). MoE kernel is typically a Grouped Matrix Multiplication (GMM), with hidden states being the left tensor, and expert weights of a specific layer being the right matrix. MoE Super Kernel introduces three key modifications:

- **Global Weight Access:** The kernel is granted pointer access to the expert weights across all L layers. Because these weights are already statically resident in the MoE device's HBM, this modification incurs zero additional memory footprint.
- **Pre-calculated Address Indexing:** The memory offset for each layer's weight tensor is pre-computed and stored in an on-device address array for constant-time lookup.
- **Dynamic Resolution:** The layer ID is treated as a dynamic device-side argument rather than a host-side compilation constant. The kernel directly indexes the address array using the input layer ID to locate the correct target tensor before executing the GMM operation.

In this way, the MoE Super Kernel is oblivious of layer ID, allowing ahead-of-time kernel dispatching and eliminating device idling between MoE kernels (Figure 10).

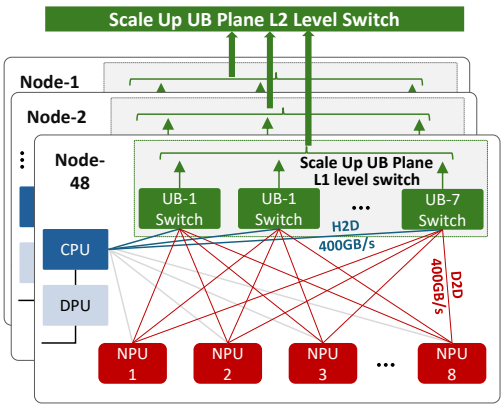


Figure 11: Hardware architecture of the CloudMatrix384 supernode. The supernode comprises 48 nodes, each integrating 8 NPUs. All 384 NPUs are interconnected via a two-level hierarchical UB plane, providing 400 GB/s bandwidth between any NPU pair.

3.5 Putting It All Together

Upon arrival, user requests are aggregated via **length-aware batching** to ensure optimal MoE token density. Once a pair of batches is formed, they are co-scheduled to an idle attention DP group using **dual-batch interleaving**. The attention devices alternate their layer-wise processing, achieving seamless **communication-computation overlapping** by immediately computing the next request batch while invoking `async-dispatch-send` of the previous batch in separate communication streams. Concurrently, MoE devices process these decoupled workloads sequentially and out-of-order via the **MoE Super Kernel**, completely eliminating host-side stalls through **bubble-free kernel dispatching**. Finally, expert results are returned via `async-combine-send`, allowing the originating attention devices to eagerly advance to the next layer. Ultimately, this orchestrated pipeline fully dismantles global DP synchronization barriers, unlocking high-throughput, asynchronous MoE inference.

4 ASAP Implementation

4.1 Hardware Platform

ASAP is implemented and deployed on the Huawei CloudMatrix384 supernode [22, 56] (Figure 11), which interconnects 384 Ascend 910 NPUs. Each NPU die has 24 cube cores, 48 vector cores and 64 GB HBM. The NPUs are linked via a Unified-Bus (UB) [28] mesh, providing a shared memory abstraction with a uniform bi-directional 400 GB/s bandwidth between any device pairs in the supernode. This architecture supports remote direct memory access with microsecond-level latency.

4.2 Serving Framework and Deployment

ASAP is integrated into a production-grade inference framework [22] based on PyTorch 2.1 and CANN 8.3 [2]. For our primary evaluation of DeepSeek-V3.2, we adopt a disaggregated Attention-MoE configuration with $D = 4$, $T = 4$, $E = 16$, totaling 32 NPUs. This configuration maintains the same device count as the standard $DP8$-$TP4$-$EP32$ synchronous setup [13]. Note that ASAP supports other combinations of parallelism degrees, especially when $D \times T \neq E$ thanks to the disaggregated architecture. The optimal values of D, T, and E can be derived from design space exploration [22, 55], which is orthogonal to our work.

Due to the HBM capacity constraints (which must host the KV cache on attention devices), we set the maximum sequence length per device to 8k. With $TP = 4$, this allows individual request batches up to 32k tokens (i.e., $S = 32k$); requests longer than $32k$ are routed to dedicated prefill instances utilizing Sequence Parallelism (SP). While our implementation and evaluation focus on these limits, ASAP's architecture is orthogonal to memory capacity and scales with available hardware resources.

4.3 ASAP Components

Asynchronous Communication Primitives: The four asynchronous communication primitives (`async-dispatch-send`, `async-dispatch-recv`, `async-combine-send`, and `async-combine-recv`) are implemented in AscendC [1] with approximately 4,000 lines of code (LoC) in total. The shared memory buffer on each device is statically allocated using `aclrtMalloc` [4]. Reads and writes from one NPU to the remote memory buffer of another are called through `DataCopy` or `DataCopyPad` interfaces [3] whose latency is at the granularity of microseconds.

MoE Super Kernel: We extended the baseline MoE GMM kernel (about 3,000 LoC) to support layer-oblivious execution. By introducing a layer-ID input tensor and a pre-calculated weight address list (roughly 200 LoC), the MoE Super Kernel can resolve memory offsets dynamically at runtime.

Triple-Stream Design: To mitigate resource contention between concurrent communication and computation, we partition hardware cores for our triple-stream design. Specifically, we allocate all 24 cube cores and 24 vector cores to the main compute stream. Each of the two communication streams is assigned 12 dedicated vector cores, which is sufficient to drive the non-compute-intensive data transfer logic.

While this static partitioning isolates execution units, residual contention persists at the shared memory hierarchy, particularly within the L2 cache and HBM bandwidth [2]. Our empirical profiling reveals that on attention devices, even with core partitioning, concurrent communication-related memory traffic interferes with compute-intensive kernels, leading to a measurable elongation of attention latency. Consequently,

we selectively deploy the triple-stream design only on MoE devices.

Host-Side Logic: We modified approximately 1,000 LoC in the request scheduler and model executor to implement length-aware batching, dual-batch interleaving, and bubble-free kernel dispatching.

4.4 Discussion

Portability Beyond Supernodes. ASAP does not strictly require supernode architectures. While UB-based load/store primitives provide ultra-low intra-node latency, the asynchronous paradigm is platform-agnostic. The logic remains robust over RDMA-based interconnects; even with RDMA latencies of hundreds of microseconds, the performance gains from dismantling global synchronization barriers (which can span tens of milliseconds) significantly outweigh the communication overhead.

Applicability to Decode Phase. While ASAP can be applied to the decode phase, the benefits are likely to be less pronounced for two reasons: (1) Complexity Scaling: Decode attention is $O(s)$ rather than $O(s^2)$, resulting in lower variance in attention latency and more manageable DP imbalance. (2) Arithmetic Intensity: The decode phase generates only one token per request. Accumulating enough tokens to saturate MoE efficiency (e.g., 2k tokens) would introduce queuing delays that may negatively affect decode SLO. Therefore, ASAP primarily focuses on the prefill phase.

5 Evaluation

5.1 Methodology

MoE Model. We evaluate ASAP using Deepseek-V3.2 [30], a state-of-the-art open-sourced MoE model [30], released in December 2025. DeepSeek-V3.2 features a massive parameter scale of 671B, with 256 experts and 1 shared expert. It activates only 37B parameters (8 experts and 1 shared expert) per token. Deepseek-V3.2 incorporates DSA, a novel sparse attention mechanism.

Model Deployment. Following the baseline deployment in Deepseek's technical report [13], one prefill instance of the synchronous baseline utilizes 32 NPU dies with $DP = 8$, $TP = 4$ and $EP = 32$. To ensure a fair comparison, ASAP employs a disaggregated $DP = 4$, $TP = 4$ and $EP = 16$ configuration, maintaining the same device count as synchronous systems. All experiments are conducted on CloudMatrix384 supernodes (detailed in Section 4.1).

Workloads. Similar to prior work [25, 47], we evaluate various request rate and lengths. We simulate realistic serving scenarios with request inter-arrival times following a Poisson distribution [43]. Request rate is quantified by request-per-second (RPS). Request lengths are sampled from a production trace dataset from *Huawei Cloud*'s, shown in Figure 5 and discussed in Section 2.3. Requests longer that 32k are excluded, as discussed in Section 4.2. We set the SLO for TTFT at 5 seconds. Each run lasts for five minutes.

Metrics. All the systems decouple prefill and decode phases onto different devices, and we evaluate only the performance of the prefill instance. We evaluate: (1) **Mean TTFT** across varying request rates (RPS), and (2) **SLO-compliant Throughput**, defined as the maximum RPS sustained without violating the 5s SLO.

Baselines. We compare ASAP against two synchronous state-of- the-art inference systems:

- **Default:** A standard scheduling and execution logic in the in-house inference framework [22], similar to vLLM [25]. The resource scheduler aggregates incoming requests into batches of similar total token counts, to balance loads between DP groups.
- **ChunkedPrefill [5]:** Long sequences are split into 8k chunks, which get processed sequentially. With $TP = 4$, this means that the maximum sequence length on each attention device is reduced from 8k to 2k. This mitigates the sequence-length variance and DP imbalance problem.

5.2 End-to-End Performance

Figure 12 shows the mean TTFT of each inference system with increasing RPS, and Figure 13 reports the normalized throughput under the 5s SLO. We observe that:

- For each inference system, mean TTFT is flat at the beginning, and increases almost exponentially after an inflection point. ASAP significantly delays this inflection point. It consistently outperforms all the synchronous baselines across the entire range of request rates. For Default, load balancing using aggregate sequence length is proven to be ineffective, as discussed in Section 2.2.1. For ChunkedPrefill, the reduced variance in sequence length indeed mitigates DP imbalance, thus outperforming Default. However, the system is still synchronous, and does not completely eliminate the DP imbalance issue.
- At a low load ($RPS = 1$) when requests rarely get queued in any of the inference systems, ASAP achieves mean TTFT of 350ms, representing a 34.3% and 9.8% reduction compared to Default (533ms) and ChunkedPrefill (388ms), respectively. As load increases, the benefits of asynchronous execution become more pronounced. At $RPS = 4$, ASAP reduces TTFT by 54.9% and 41.8% over Default and ChunkedPrefill, respectively. At $RPS = 8$, Default is unable to sustain the request rate, and ASAP reduces mean TTFT by 68.3% over ChunkedPrefill.
- ASAP sustains an RPS of 20, whereas Default and ChunkedPrefill reach their limits at 6.8 and 10.5 RPS, respectively. This translates to **194%** and **90%** throughput improvement, demonstrating ASAP's superior ability to ab-

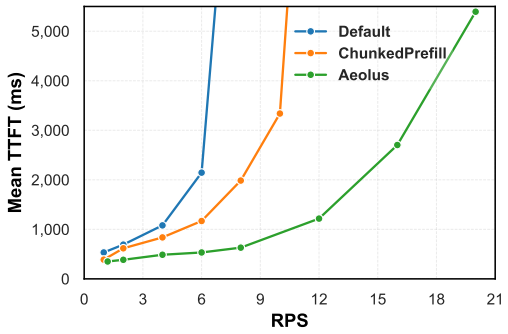


Figure 12: Mean TTFT with increasing request-per-second (RPS).

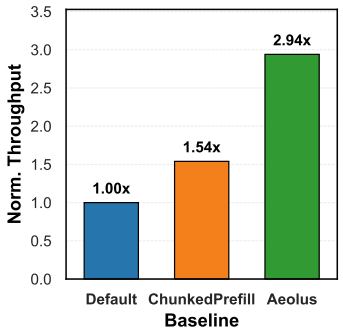


Figure 13: Normalized throughput under SLO.

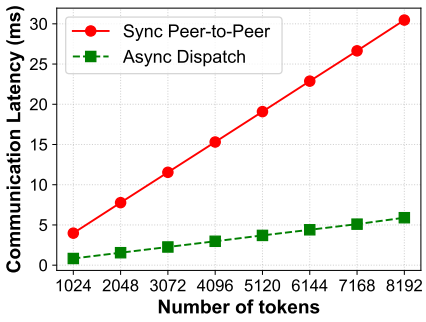


Figure 14: Communication latency with increasing token count.

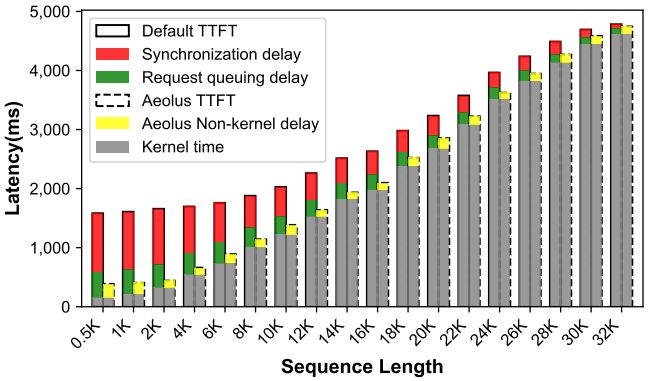


Figure 15: Latency decomposition at QPS=4. The left and right bars at each sequence length show mean TTFT under Default and ASAP, respectively. For Default, TTFT is decomposed into kernel time, request queuing delay, and synchronization waiting delay. For ASAP, TTFT is decomposed into kernel and non-kernel delay.

sorb heterogeneous workloads that traditionally trigger severe synchronization stalls.

5.3 Latency Decomposition

To understand the source of the performance gains, Figure 15 decomposes the TTFT of Default and ASAP at RPS=4. for Default, TTFT is decomposed into kernel time (the actual computation latency), request queuing delay (waiting time after request arrival and before request processing), and synchronization waiting delay (due to DP imbalance). For ASAP, due to the small value, we do not further decompose non-kernel latency which mainly comes from queuing on attention devices due to dual-batch interleaving. Each bar represents a range of requests. For example, the first and second group of bars in Figure 15 represent mean latency of requests under 512 tokens, and requests between 512 and 1024 tokens, respectively.

Latency increases with request sequence length due to increasing kernel time. Kernel time is the same under the two systems. However, in synchronous systems (Default), non-kernel overheads dominate for short sequences. For instance, for requests under 512 tokens, synchronization delay and request queuing delay account for 55% and 30% of the overall TTFT. This confirms the "straggler effect" where short requests are penalized the most by global barriers.

In contrast, ASAP reduces non-kernel delay up to **80%** for the same sequence range. By allowing request batches to "flow" asynchronously, ASAP effectively converts idle synchronization time into productive computation intervals. As sequence length increases, the relative impact of synchronization diminishes, yet ASAP maintains a consistent edge by eliminating residual execution bubbles.

5.4 Communication Efficiency

The foundation of ASAP is its non-blocking, asymmetric and asynchronous communication. If not designing a new set of primitives, one has to use existing synchronous peer-to-peer (P2P) communication to realize asymmetric communication between attention and MoE devices.

For instance, after attention computation, if the attention device needs to dispatch its tokens to n MoE devices. Then, the P2P communication will be invoked n times, Each time, the attention device (i.e., sender) has to handshake with each MoE device (i.e., receiver), and the data transfer will not be done until the receiver has received the data transfer. If the receiver is doing its own computation task, the sender will be blocked until the receiver finished the ongoing task and acknowledged the data transfer. Therefore, in p2p communication, the communication latency include (1) the handshake time which is a constant, (2) the data transfer time which is linear to data amount, (3) extra receiving delay which fluctuates depending on the status of each receiver, and (4) the total communication latency is n times of the communication latency between the sender and each receiver. On the contrary, ASAP's `async-dispatch-send` is non-blocking, so it doesn't need to handshake with the receiver. It transfers data and immediately returns back future tasks. There is no extra receiving delay either because each receiver performs

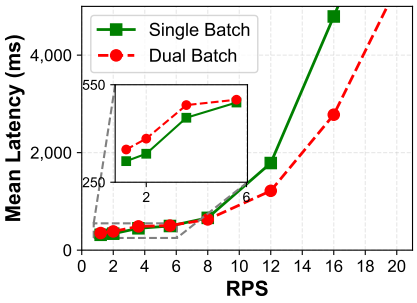


Figure 16: Comparison of mean TTFT with/without dual-batch interleaving.

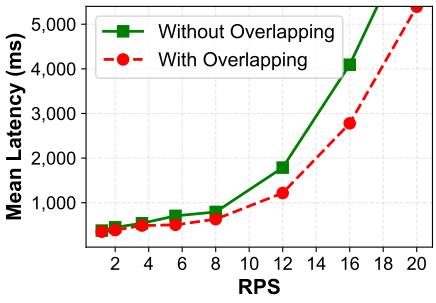


Figure 17: Comparison of mean TTFT with/without communication-computation overlapping.

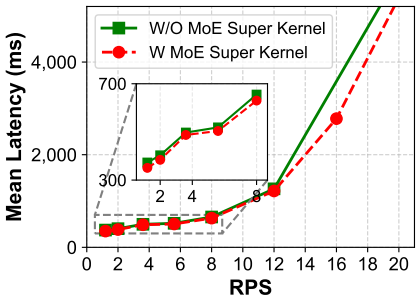


Figure 18: Comparison of mean TTFT with/without the bubble-free kernel dispatching brought by MoE Super Kernel.

`asyn-dispatch-recv` whenever it's ready, without blocking the sender. Also, the sender can perform data transfer to n devices at the same time in parallel.

Figure 14 compares the communication latency of the synchronous P2P and the asynchronous `async-dispatch`, with an increasing number of tokens. Both curves show a linear trend with token count. Sending t tokens implies sending data size of $t * H * E_{total}/E$. For DeepSeek V3.2, this is 63MB per 1k tokens. We can see that latency of synchronous p2p is 4× of `async-dispatch` under 1k tokens, and 5.8× under 8k tokens. The difference increases with more data.

Note that the latency of our asynchronous communication primitive by leveraging supernode is significantly lower than that without support of the high-bandwidth peer-to-peer network in supernodes. Prior work [33] develops a set of asynchronous p2p primitives on GPUs leveraging the IBGDA Connection. The reported communication latency to send 512 tokens is over 0.5ms. However, the latency of our primitive is less than 0.1ms under the same token count.

5.5 Ablation Studies

We perform a series of ablation studies to show the effectiveness of each mechanism in ASAP.

5.5.1 Effectiveness of Dual-Batch Interleaving

Figure 16 evaluates the impact of dual-batch interleaving on mean TTFT across varying request rates. At low RPS, where request arrivals are sparse, the dual-batch mechanism introduces a marginal TTFT increase of approximately 6.5%. This overhead stems from the difference in attention and MoE latency, as illustrated in Figure 8: if the attention latency of one batch (B_1) exceeds the MoE latency of the co-scheduled batch (B_2), B_2 must wait for the attention devices to become available, introducing minor stalls.

However, as the RPS increases, the system without interleaving suffers from frequent attention-stage idling during MoE offloading, leading to rapid queue accumulation. By contrast, ASAP's dual-batch interleaving effectively reclaims these execution gaps, enabling the system to sustain high request rate under latency constraints. Consequently, it improves the SLO-compliant throughput by **14.3%** (from 17.5 to 20 RPS). This result underscores that dual-batch interleaving is essential for maximizing attention device duty cycles and maximizing the overall system throughput.

5.5.2 Effectiveness of Communication-Computation Overlapping

The efficacy of our triple-stream design in masking communication overhead is analyzed in Figure 17. In low-load scenarios, overlapping yields negligible TTFT reduction as the system operates with ample resource headroom. As the request rate intensifies, the overlapping becomes critical bottleneck, which increases the SLO-compliant throughput from 17.8 to 20, an **12.4%** improvement.

5.5.3 Effectiveness of Bubble-Free Kernel Dispatching

Figure 18 quantifies the gains from the MoE Super Kernel that enables bubble-free kernel dispatching. At low RPS, the bubble-free kernel dispatching reduces TTFT by roughly 13ms. This is because the traditional kernel dispatching latency from the host CPU is 220 microseconds each layer measured on the CloudMatrix384 supernode. Since there are 61 layers in DeepSeek V3.2, this translates to $220 * 61 \approx 13.4ms$ saving in TTFT. The latency saving also brings higher throughput. Compared to the systems without the MoE Super Kernel, our bubble-free kernel dispatching increases the SLO-compilant throughput by 6%.

6 Related Work

Long-context LLM Inference. Extensive research has focused on accelerating attention computation for long sequences. FlashAttention [12] and FlashDecoding [20] optimize memory I/O through tiling and recomputation, though they do not alter the quadratic complexity $O(s^2)$ to request sequence length s in the prefill phase. Other approaches, such

as sparse attention [9] and linear attention [24], attempt to reduce this complexity. Our characterization (Section 2) and evaluation (Section 5) have already applied sparse attention, and we demonstrate the necessity and benefits of ASAP's asynchronous execution.

Sequence Parallelism (SP) [6, 26, 32] partitions long sequences across devices to enable concurrent processing.

LoongServe [47] and Infinite-LLM [29] further refine this by dynamically adjusting SP degrees or dedicating specialized clusters for long-context requests. Additionally, ChunkedPrefill [5] and BucketServe [52] mitigate request variance by decomposing long prompts or grouping similar-length requests. While these techniques are orthogonal and complementary to ASAP, they remain within the synchronous execution paradigm; they merely alleviate rather than eliminate the idle bubbles caused by intrinsic workload variance.

Disaggregated LLM Serving Systems. The computational disparity between different inference phases has led to the rise of disaggregated architectures. Prefill-decode disaggregation, exemplified by DistServe [53], Splitwise [37], and Aegaeon [48], separates the compute-bound prefill and memory-bound decode stages to minimize interference. Mooncake [38] extends this by pooling CPU and DRAM resources for a distributed KV cache. However, these systems primarily focus on inter-phase isolation and do not address the intra-phase latency fluctuations driven by dynamic workloads.

Recently, Attention-MoE disaggregation has been proposed to enable independent scaling of attention and MoE modules, including frameworks such as MegaScale-Infer [55] and Step-3 [44]. However, they still rely on synchronous barriers across attention DP groups and do not tackle the DP imbalance dilemma. Furthermore, these frameworks are predominantly tailored for the decoding stage. In contrast, ASAP specifically targets the prefill stage, introducing an asynchronous execution pipeline to tackle resource idling due to DP imbalance.

Asynchronous Inference Systems. Expert-as-a-Service (EaaS) [33] designs a CPU-free asynchronous peer-to-peer communication library for MoE serving. However, it mainly targets scaling at the granularity of each expert, and does not tackle DP imbalance. The communication library includes only peer-to-peer primitives, and does not provide asymmetric 1-to-N/N-to-1 communication primitives. On the contrary, ASAP adopts a set of efficient, asynchronous, and asymmetric communication primitives. Also, the communication latency of our primitives is much smaller than that reported in EaaS by taking advantage of supernodes.

7 Conclusion

Existing synchronous inference systems introduce costly barriers under DP imbalance in online MoE serving. We present ASAP, the first asynchronous inference system to speed up the prefill phase of MoE models. Experiments show that ASAP improves throughput under SLO by 90% over the state-of-the-art inference systems.

References

[1] Ascend C Custom Operator Development and Usage. https://www.mindspore.cn/tutorials/experts/en/r2.3.1/operation/op_custom_ascendc.html, 2025.

[2] Compute Architecture for Neural Networks (CANN). https://www.hiascend.com/cann, 2025.

[3] Data Movement in Ascend C. https://www.hiascend.com/document/detail/zh/canncommercial/83RC1/API/ascendcopapi/atlasascendc_api_07_0102.html, 2025.

[4] Memory Management of Ascend NPU. https://www.hiascend.com/document/detail/zh/canncommercial/83RC1/API/appdevgapi/aclpythondevg_01_0110.html, 2025.

[5] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*, 2023.

[6] William Brandon, Aniruddha Nrusimha, Kevin Qian, Zachary Ankner, Tian Jin, Zhiye Song, and Jonathan Ragan-Kelley. Striped attention: Faster ring attention for causal transformers. *arXiv preprint arXiv:2311.09431*, 2023.

[7] Jingwei Cai, Dehao Kong, Hantao Huang, Zishan Jiang, Zixuan Ma, Qingyu Guo, Zhenxing Zhang, Guiming Shi, Mingyu Gao, and Kaisheng Ma. Characterizing cloud-native llm inference at bytedance and exposing optimization challenges and opportunities for future ai accelerators. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 1–19. IEEE, 2026.

[8] Shiheng Cao, Junmin Wu, Junshi Chen, Hong An, and Zhibin Yu. Amali: An analytical model for accurately modeling llm inference on modern gpus. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ISCA '25, page 1495–1508, New York, NY, USA, 2025. Association for Computing Machinery.

[9] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

[10] Yujeong Choi, Yunseong Kim, and Minsoo Rhu. Lazy batching: An sla-aware batching system for cloud machine learning inference. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 493–506, 2021.

[11] Peizhuang Cong, Aomufei Yuan, Shimao Chen, Yuxuan Tian, Bowen Ye, and Tong Yang. Prediction is all moe needs: Expert load distribution goes from fluctuating to stabilizing, 2024.

[12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *arXiv preprint arXiv:2205.14135*, 2022.

[13] DeepSeek-AI, Aixin Liu, Bei Feng, and etc Bing Xue. Deepseek-v3 technical report, 2025.

[14] Jiayu Ding, Shuming Ma, Li Dong, Xingxing Zhang, Shaohan Huang, Wenhui Wang, Nanning Zheng, and Furu Wei. Longnet: Scaling transformers to 1,000,000,000 tokens, 2023.

[15] Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten Bosma, Zongwei Zhou, Tao Wang, Yu Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathleen Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc V Le, Yonghui Wu, Zhifeng Chen, and Claire Cui. Glam: Efficient scaling of language models with mixture-of-experts. *arXiv preprint arXiv:2112.06905*, 2022.

[16] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: scaling to trillion parameter models with simple and efficient sparsity. *J. Mach. Learn. Res.*, 23(1), January 2022.

[17] Ze-Feng Gao, Peiyu Liu, Wayne Xin Zhao, Zhong-Yi Lu, and Ji-Rong Wen. Parameter-efficient mixture-of-experts architecture for pre-trained language models. *arXiv preprint arXiv:2203.01104*, 2022.

[18] Raja Gond, Nipun Kwatra, and Ramachandran Ramjee. Tokenweave: Efficient compute-communication overlap for distributed llm inference. *arXiv preprint arXiv:2505.11329*, 2025.

[19] Ruihao Gong, Shihao Bai, Siyu Wu, Yunqian Fan, Zaijun Wang, Xiuhong Li, Hailong Yang, and Xianglong Liu. Past-future scheduler for llm serving under sla guarantees. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS '25, page 798–813, New York, NY, USA, 2025. Association for Computing Machinery.

[20] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Yuhan Dong, and Yu Wang. Flashdecoding++: Faster large language model inference on gpus. *arXiv preprint arXiv:2311.01282*, 2024.

[21] Cunchen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yungang Bao, and Ninghui Sun. Memserve: Context caching for disaggregated llm serving with elastic memory pool. *arXiv preprint arXiv:2406.17565*, 2024.

[22] Junhao Hu, Jiang Xu, Zhixia Liu, Yulong He, Yuetao Chen, Hao Xu, Jiang Liu, Jie Meng, Baoquan Zhang, Shining Wan, Gengyuan Dan, Zhiyu Dong, Zhihao Ren, Changhong Liu, Tao Xie, Dayun Lin, Qin Zhang, Yue Yu, Hao Feng, Xusheng Chen, and Yizhou Shan. Deepserve: serverless large language model serving at scale. In *Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference*, USENIX ATC '25, USA, 2025. USENIX Association.

[23] Yunpeng Huang, Jingwei Xu, Junyu Lai, Zixu Jiang, Taolue Chen, Zenan Li, Yuan Yao, Xiaoxing Ma, Lijuan Yang, Hao Chen, Shupeng Li, and Penghao Zhao. Advancing transformer architecture in long-context large language models: A comprehensive survey, 2024.

[24] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.

[25] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP '23, page 611–626, New York, NY, USA, 2023. Association for Computing Machinery.

[26] Dacheng Li, Rulin Shao, Anze Xie, Eric Xing, Joseph Gonzalez, Ion Stoica, Xuezhe Ma, and Hao Zhang. Lightseq: : Sequence level parallelism for distributed training of long context transformers. In *Workshop on Advancing Neural Network Training: Computational Efficiency, Scalability, and Resource Optimization (WANT@NeurIPS 2023)*, 2023.

[27] Shenggui Li, Fuzhao Xue, Chaitanya Baranwal, Yongbin Li, and Yang You. Sequence parallelism: Long sequence training from system perspective. *arXiv preprint arXiv:2105.13120*, 2022.

[28] Heng Liao, Bingyang Liu, Xianping Chen, Zhigang Guo, Chuanning Cheng, Jianbing Wang, Xiangyu Chen, Peng

Dong, Rui Meng, Wenjie Liu, Zhe Zhou, Ziyang Zhang, Yuhang Gai, Cunle Qian, Yi Xiong, Zhongwu Cheng, Jing Xia, Yuli Ma, Xi Chen, Wenhua Du, Shizhong Xiao, Chungang Li, Yong Qin, Liudong Xiong, Zhou Yu, Lv Chen, Lei Chen, Buyun Wang, Pei Wu, Junen Gao, Xiaochu Li, Jian He, Shizhuan Yan, and Bill McColl. Ub-mesh: A hierarchically localized nd-fullmesh data center network architecture. *IEEE Micro*, 45(5):20–29, 2025.

[29] Bin Lin, Chen Zhang, Tao Peng, Hanyu Zhao, Wencong Xiao, Minmin Sun, Anmin Liu, Zhipeng Zhang, Lanbo Li, Xiafei Qiu, Shen Li, Zhigang Ji, Tao Xie, Yong Li, and Wei Lin. Infinite-llm: Efficient llm service for long context with distattention and distributed kvcache. *arXiv preprint arXiv:2401.02669*, 2024.

[30] Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, and Chen Dong. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.

[31] Guowei Liu, Hongming Li, Yaning Guo, Yongxi Lyu, Mo Zhou, Yi Liu, Zhaogeng Li, and Yanpeng Wang. Revealing the challenges of attention-ffn disaggregation for modern moe models and hardware systems. *arXiv preprint arXiv:2602.09721*, 2026.

[32] Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:310.01889*, 2023.

[33] Ziming Liu, Boyu Tian, Guoteng Wang, Zhen Jiang, and etc Peng Sun. Expert-as-a-service: Towards efficient, scalable, and robust large-scale moe serving. *arXiv preprint arXiv:2509.17863*, 2025.

[34] Haiyue Ma, Zhixu Du, and Yiran Chen. Moe-gps: Guidlines for prediction strategy for dynamic expert duplication in moe load balancing, 2025.

[35] Siyuan Mu and Sen Lin. A comprehensive survey of mixture-of-experts: Algorithms, theory, and applications. *arXiv preprint arXiv:2503.07137*, 2025.

[36] Rui Pan, Zhuang Wang, Zhen Jia, Can Karakus, Luca Zancato, Tri Dao, Yida Wang, and Ravi Netravali. Marconi: Prefix caching for the era of hybrid llms. In M. Zaharia, G. Joshi, and Y. Lin, editors, *Proceedings of Machine Learning and Systems*, volume 7. MLSys, 2025.

[37] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture*, ISCA '24, page 118–132. IEEE Press, 2025.

[38] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 155–170, Santa Clara, CA, February 2025. USENIX Association.

[39] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato, editors, *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 18332–18346. PMLR, 17–23 Jul 2022.

[40] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2020.

[41] Yehui Tang, Xiaosong Li, Fangcheng Liu, Wei Guo, Hang Zhou, Yaoyuan Wang, Kai Han, Xianzhi Yu, Jinpeng Li, Hui Zang, Fei Mi, Xiaojun Meng, Zhicheng Liu, Hanting Chen, Binfan Zheng, Can Chen, Youliang Yan, Ruiming Tang, Peifeng Qin, Xinghao Chen, Dacheng Tao, and Yunhe Wang. Pangu pro moe: Mixture of grouped experts for efficient sparsity. *arXiv preprint arXiv:505.21411*, 2025.

[42] Kimi Team, Yifan Bai, Yiping Bao, and etc Guanduo Chen. Kimi k2: Open agentic intelligence, 2025.

[43] W.J. Thompson. Poisson distributions. *Computing in Science and Engineering*, 3(3):78–82, 2001.

[44] Bin Wang, Bojun Wang, Changyi Wan, Guanzhe Huang, Hanpeng Hu, Haonan Jia, Hao Nie, Mingliang Li, Nuo Chen, and Siyu Chen. Step-3 is large yet affordable: Model-system co-design for cost-effective decoding. *arXiv preprint arXiv:2507.19427*, 2025.

[45] Yujie Wang, Shiju Wang, Shenhan Zhu, Fangcheng Fu, Xinyi Liu, Xuefeng Xiao, Huixia Li, Jiashi Li, Faming Wu, and Bin Cui. Flexsp: Accelerating large language model training via flexible sequence parallelism. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS '25, page 421–436, New York, NY, USA, 2025. Association for Computing Machinery.

[46] Zheng Wang, Anna Cai, Xinfeng Xie, Zaifeng Pan, Yue Guan, Weiwei Chu, Jie Wang, Shikai Li, Jianyu Huang, and Chris Cai. {WLB-LLM}:{Workload-Balanced} 4d parallelism for large language model training. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 785–801, 2025.

[47] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, SOSP '24, page 640–654, New York, NY, USA, 2024. Association for Computing Machinery.

[48] Yuxing Xiang, Xue Li, Kun Qian, Yufan Yang, Diwen Zhu, Wenyuan Yu, Ennan Zhai, Xuanzhe Liu, Xin Jin, and Jingren Zhou. Aegaeon: Effective gpu pooling for concurrent llm serving on the market. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, pages 1030–1045, 2025.

[49] Yuxing Xiang, Xue Li, Kun Qian, Wenyuan Yu, Ennan Zhai, and Xin Jin. Servegen: Workload characterization and generation of large language model serving in production. *arXiv preprint arXiv:2505.09999*, 2025.

[50] An Yang, Anfeng Li, and etc Baosong Yang. Qwen3 technical report, 2025.

[51] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: efficient execution of structured language model programs. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA, 2024. Curran Associates Inc.

[52] Wanyi Zheng, Minxian Xu, Shengye Song, and Kejiang Ye. Bucketserve: Bucket-based dynamic batching for smart and efficient llm inference serving. *arXiv preprint arXiv:2507.17120*, 2025.

[53] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, 2024.

[54] Qianchao Zhu, Jiangfei Duan, Chang Chen, Siran Liu, Xiuhong Li, Guanyu Feng, Xin Lv, Xiao Chuanfu, Dahua Lin, and Chao Yang. Sampleattention: Near-lossless acceleration of long context llm inference with adaptive structured sparse attention. In M. Zaharia, G. Joshi, and Y. Lin, editors, *Proceedings of Machine Learning and Systems*, volume 7. MLSys, 2025.

[55] Ruidong Zhu, Ziheng Jiang, Chao Jin, Peng Wu, Cesar A Stuardo, Dongyang Wang, Xinlei Zhang, Huaping Zhou, Haoran Wei, and Yang Cheng. Megascale-infer: Efficient mixture-of-experts model serving with disaggregated expert parallelism. In *Proceedings of the ACM SIGCOMM 2025 Conference*, pages 592–608, 2025.

[56] Pengfei Zuo, Huimin Lin, Junbo Deng, Nan Zou, Xingkun Yang, Yingyu Diao, Weifeng Gao, Ke Xu, Zhangyu Chen, and Shirui Lu. Serving large language models on huawei cloudmatrix384. *arXiv preprint arXiv:2506.12708*, 2025.